Sprint 1 Notes: React Orientation, SPAs, and Vite Setup

Sprint Goal

By the end of Sprint 1, you should understand three main things:

1. Why React exists.
2. What a Single Page Application, or SPA, is.
3. How to create and run your first React project using Vite.

Sprint 1 is not about advanced React. It is about getting comfortable with the basic ideas and creating your first working React app.

---

1. What Is React?

React is a JavaScript library used to build user interfaces.

A user interface is the part of an app or website that users see and interact with.

Examples of user interface parts include:

- Buttons
- Forms
- Cards
- Menus
- Navigation bars
- Dashboards
- Pages
- Search boxes
- Login screens

React helps you build these visible parts in a cleaner and more reusable way.

Simple Definition

React helps developers build websites and apps by splitting the user interface into small reusable pieces.

These small pieces are called components.

---

2. What Are Components?

A component is a small reusable part of a user interface.

Instead of building one huge page all in one file, React encourages you to split the page into smaller parts.

For example, a page may be made of these components:

```
App
 ├── Header
 ├── ProfileCard
 ├── Button
 └── Footer
```

Each component has its own job.

Example:

- `Header` shows the top part of the page.
- `ProfileCard` shows user information.
- `Button` shows a clickable button.
- `Footer` shows the bottom part of the page.

Why Components Are Useful

Components are useful because they make your code:

- Easier to read
- Easier to reuse
- Easier to test
- Easier to fix
- Easier to organize

Instead of repeating the same code many times, you can create one component and use it wherever you need it.

---

3. React as a UI Library

React is usually called a UI library.

UI means user interface.

React mainly focuses on building the view layer of an application.

The view layer means the part of the app that the user sees.

React helps answer this question:

```
What should the screen look like right now?
```

React does not force you to use one specific full app structure. You can choose other tools for routing, styling, testing, and data fetching.

That is one reason React is considered flexible.

---

4. Library vs Framework

A library and a framework are both tools developers use, but they are not exactly the same.

Library

A library gives you tools.

You decide when and how to use those tools.

Simple idea:

```
Library:
"Here are useful tools. You control the app structure."
```

React is a library because it mainly gives you tools for building user interfaces.

Framework

A framework gives you tools and also gives you a bigger structure.

The framework often decides how the app should be organized.

Simple idea:

```
Framework:
"Here are tools, and here is the structure you should follow."
```

Next.js is a framework because it adds many things on top of React, such as:

- Routing
- Server rendering
- API routes
- Image optimization
- App-level structure
- Data fetching patterns

Simple Comparison

Topic	Library	Framework
Control	You control more decisions	The framework controls more decisions
Structure	Flexible	More opinionated
Example	React	Next.js
Main role	Gives tools	Gives tools plus structure

---

5. Why React Exists

React exists to make user interface development easier.

In older or plain JavaScript projects, developers often manually select HTML elements and update them.

Example using plain JavaScript:

```
const countElement = document.querySelector("#count");
countElement.textContent = count;
```

This means the developer has to manually find an element and change it.

React uses a different way of thinking.

Instead of saying:

```
Find this element and manually change it.
```

React encourages you to say:

```
Describe what the UI should look like for the current data.
```

This makes React apps more data-driven.

Beginner idea:

```
Data changes
↓
React updates the UI
```

You will learn more about this later when you study state.

For Sprint 1, remember this:

React helps you build UI by describing what should appear on the screen.

---

6. What Is an SPA?

SPA means Single Page Application.

A Single Page Application loads one main HTML page. After that, JavaScript changes the visible content without reloading the whole page.

Traditional Website Flow

A traditional website usually works like this:

```
User clicks a link
↓
Browser asks the server for a new HTML page
↓
Server sends a new page
↓
Browser reloads the full page
```

This means every page change may cause a full reload.

SPA Flow

An SPA usually works like this:

```
User clicks a link
↓
JavaScript changes what appears on the page
↓
The browser does not reload the full page
```

This can make the app feel faster and smoother.

---

7. Why SPAs Can Feel Fast

SPAs can feel fast because the browser does not need to reload the whole page for every screen change.

Example:

If a user clicks from `Home` to `Dashboard`, a React SPA can change only the visible component instead of asking the server for a completely new HTML page.

This creates a smoother user experience.

Simple Mental Model

```
Traditional website:
New page = full reload

SPA:
New view = JavaScript changes the screen
```

---

8. SPA Risks

SPAs are useful, but they can create problems if they are not handled carefully.

Sprint 1 introduces these risks so you understand them early.

You do not need to solve all of them in Sprint 1.

Risk 1: Accessibility Problems

Accessibility means making apps usable for people with different needs, including people who use screen readers or keyboard navigation.

In an SPA, content can change dynamically without a full page reload.

A screen reader may not automatically understand that the page content changed.

Beginner Solution

Use good accessibility habits, such as:

- Semantic HTML
- Proper headings
- Accessible routing
- Focus management
- Real buttons for actions

You will learn more about accessibility later.

Risk 2: Navigation Problems

In a normal website, each page has its own URL.

In an SPA, JavaScript changes the view. Because of that, browser navigation needs to be handled carefully.

Possible problems:

- Back button may not work correctly.
- Forward button may not work correctly.
- Bookmarked pages may not open the expected view.
- Direct URLs may not work correctly.

Beginner Solution

Use a routing library such as React Router.

React Router helps connect URLs to React components.

You will learn routing in a later sprint.

Risk 3: SEO Problems

SEO means Search Engine Optimization.

It helps search engines understand and rank your pages.

Some SPAs can be harder for search engines to read because the content may be generated by JavaScript after the initial page load.

Beginner Solution

Common solutions include:

- Server-side rendering, or SSR
- Pre-rendering
- SEO-friendly URLs
- Using frameworks like Next.js when SEO is important

You do not need to master these in Sprint 1.

For now, just remember that SPAs need extra care for SEO.

Risk 4: Slow Initial Load

An SPA may need to load a large JavaScript bundle at the start.

If the bundle is too large, the first page load can be slow.

After the first load, navigation may feel fast, but the first load still matters.

Beginner Solution

Later, you can reduce this problem with:

- Code splitting
- Lazy loading
- Smaller bundles
- Performance optimization

For Sprint 1, only remember the risk.

---

9. SPA Benefits and Risks Summary

Topic	Explanation
SPA meaning	Single Page Application
Main idea	One HTML page loads, then JavaScript updates the view
Main benefit	Smooth navigation without full page reloads
Main risk 1	Accessibility issues
Main risk 2	Navigation and bookmark issues
Main risk 3	SEO issues
Main risk 4	Slow initial load if JavaScript is large

---

10. React App Boot Flow

A React app starts from a few important files.

The beginner boot flow looks like this:

```
index.html
↓
main.jsx
↓
App.jsx
↓
components
↓
screen
```

What This Means

`index.html`

This is the main HTML page.

In an SPA, this is usually the single HTML page that loads first.

`main.jsx`

This file starts the React app.

It connects React to the HTML page.

`App.jsx`

This is usually the main starter component.

In Sprint 1, this is the file you usually edit first.

Components

Components are the smaller UI pieces used inside the app.

You will learn more about components in Sprint 2.

---

11. What Is Vite?

Vite is a modern frontend build tool.

It helps you create, run, and develop React projects.

In simple words:

```
Vite helps you quickly start a React project on your computer.
```

Vite gives you:

- A starter project structure
- A development server
- Fast refresh while editing
- Build tools for modern frontend development

Why Beginners Use Vite

Vite is useful because it lets you start coding React without manually setting up many build tools.

Instead of creating every file yourself, you can run one command and get a working React starter project.

---

12. Before Using Vite

Before creating a Vite React app, you need Node.js installed.

Node.js lets you run JavaScript tools outside the browser.

npm comes with Node.js.

npm is the Node package manager.

You use npm to:

- Create projects
- Install dependencies
- Run project scripts

---

13. Create a React Project with Vite

To create your first Vite React project, open your terminal and run:

```
npm create vite@latest my-react-app -- --template react
cd my-react-app
npm install
npm run dev
```

These commands create and run your React app.

---

14. Command Breakdown

Command 1

```
npm create vite@latest my-react-app -- --template react
```

This creates a new Vite project using the React template.

Breakdown

Part	Meaning
`npm`	Node package manager
`create`	Runs a project creation tool
`vite@latest`	Uses the latest Vite project creator
`my-react-app`	The name of your project folder
`-- --template react`	Tells Vite to use the React template

Command 2

```
cd my-react-app
```

This moves you into the project folder.

`cd` means change directory.

Command 3

```
npm install
```

This installs the packages listed in the project.

These packages are called dependencies.

They are stored in a folder called `node_modules`.

Command 4

```
npm run dev
```

This starts the local development server.

A development server lets you view your app in the browser while you are building it.

Usually, Vite shows a local address like this:

```
http://localhost:5173/
```

Open that address in your browser to see the app.

---

15. Important Project Files and Folders

When Vite creates a React project, you will see several files and folders.

Sprint 1 mainly focuses on the beginner-level files.

`src/App.jsx`

This is the main component you usually edit first.

In Sprint 1, most of your work happens here.

`package.json`

This file stores project information.

It includes:

- Project name
- Scripts
- Dependencies
- Development dependencies

When you run `npm run dev`, npm looks inside `package.json` for the `dev` script.

`node_modules`

This folder contains installed packages.

You usually do not edit this folder manually.

It is created when you run:

```
npm install
```

---

16. Your First React Edit

After creating the project, open this file:

```
src/App.jsx
```

Replace the starter content with this:

```
export default function App() {
  return (
    <main>
      <h1>React Roadmap</h1>
      <p>I am learning React step by step.</p>
    </main>
  );
}
```

Save the file.

The browser should update automatically.

This is your first simple React component.

---

17. Understanding the First Component

Look at this code:

```
export default function App() {
  return (
    <main>
      <h1>React Roadmap</h1>
      <p>I am learning React step by step.</p>
    </main>
  );
}
```

`function App()`

This creates a function named `App`.

In React, a component can be a function.

`return (...)`

This returns the UI that should appear on the screen.

`<main>`

This is a semantic HTML element.

It means the main content of the page.

`<h1>`

This is the main heading.

`<p>`

This is a paragraph.

`export default`

This makes the `App` component available to be used by another file.

You will learn exports and imports more deeply in Sprint 2.

---

18. Sprint 1 Mini Project: React Roadmap Page

The Sprint 1 mini project is a one-screen page called React Roadmap.

The goal is not to build a beautiful design.

The goal is to practice:

- Editing `App.jsx`
- Writing simple JSX
- Creating a clear page structure
- Explaining React in simple words
- Understanding that React can update UI without full reloads

Use this code in `src/App.jsx`:

```
export default function App() {
  return (
    <main>
      <h1>React Roadmap</h1>
      <p>
        React helps us build user interfaces using reusable components.
      </p>

      <section>
        <h2>Core Topics</h2>

        <article>
          <h3>Components</h3>
          <p>Small reusable pieces of UI.</p>
        </article>

        <article>
          <h3>State</h3>
          <p>Data that changes and updates the screen.</p>
        </article>

        <article>
          <h3>Routing</h3>
          <p>Moving between different views in a single-page app.</p>
        </article>
      </section>

      <section>
        <h2>SPA Warning</h2>
        <p>
          SPAs can have SEO, accessibility, navigation, and first-load
performance
          issues if they are not built carefully.
        </p>
      </section>
    </main>
  );
}
```

---

19. What This Mini Project Teaches

`React Roadmap`

This heading tells the user what the page is about.

Core Topics

The page introduces three major React topics:

1. Components
2. State
3. Routing

You do not need to master these yet.

Sprint 1 only introduces them.

Components

Components are reusable pieces of UI.

Example:

```
A card, button, navbar, or form can become a component.
```

State

State is data that changes and affects the screen.

Example:

```
A counter number changes when a button is clicked.
```

You will study state properly in a later sprint.

Routing

Routing means moving between different views or pages in an app.

Example:

```
/home
/about
/dashboard
```

In React SPAs, routing is usually handled by a library like React Router.

You will study routing later.

---

20. Debug Task

The Sprint 1 debug task gives this broken command:

```
npm create vite my-react-app template react
cd my-react-app
npm install
npm dev
```

This command has mistakes.

Correct Version

```
npm create vite@latest my-react-app -- --template react
cd my-react-app
npm install
npm run dev
```

---

21. What Was Wrong in the Broken Command?

Mistake 1

Broken version:

```
npm create vite my-react-app template react
```

Correct version:

```
npm create vite@latest my-react-app -- --template react
```

The broken version does not correctly specify the latest Vite creator and the React template option.

You should include:

```
vite@latest
```

and:

```
-- --template react
```

Mistake 2

Broken version:

```
npm dev
```

Correct version:

```
npm run dev
```

Why?

Because `dev` is a script inside `package.json`.

npm scripts are usually run like this:

```
npm run script-name
```

So the correct command is:

```
npm run dev
```

---

22. Warm-Up Drills

Use these drills to check if you understand Sprint 1.

Drill 1: Create a Folder Name

Choose a valid project folder name.

Example:

```
my-react-app
```

Other examples:

```
react-roadmap
sprint-one-app
first-react-project
```

Use lowercase words and hyphens for clean folder names.

Drill 2: Write the Vite Command from Memory

Try to write this without looking:

```
npm create vite@latest my-react-app -- --template react
```

Then continue with:

```
cd my-react-app
npm install
npm run dev
```

Drill 3: Define an SPA in One Sentence

Example answer:

```
An SPA is a web app that loads one main HTML page and uses JavaScript to update
the view without full page reloads.
```

---

23. Guided Coding Task

Your guided task is:

1. Create a Vite React project.
2. Open `src/App.jsx`.
3. Replace the starter UI with one heading and one paragraph.

4. Run the app in the browser.

Step-by-Step

Run:

```
npm create vite@latest my-react-app -- --template react
cd my-react-app
npm install
npm run dev
```

Open:

```
src/App.jsx
```

Replace the content with:

```
export default function App() {
  return (
    <main>
      <h1>My First React App</h1>
      <p>This app was created with Vite.</p>
    </main>
  );
}
```

Save the file.

Open the local server URL in your browser.

---

24. Build Something Small

Build a one-screen React Roadmap page with:

- One heading
- One short intro paragraph
- Three cards
- One warning box about SPA risks

Required Cards

Your three cards should be:

1. Components
2. State
3. Routing

Example Content

```
Components: Small reusable pieces of UI.
State: Data that changes and updates the screen.
Routing: Moving between different views in an app.
```

---

25. Challenge Task

Add a short warning box that lists two SPA risks and one way to reduce each risk.

Example:

SPA Risk	How to Reduce It
SEO issues	Use SSR, pre-rendering, or a framework like Next.js
Navigation issues	Use React Router

You can also mention accessibility or slow initial loading.

---

26. Expected Output

By the end of the Sprint 1 practice, your app should meet these checks:

- The app runs in the browser.
- The app opens at a local Vite URL like `http://localhost:5173/`.
- `src/App.jsx` has been edited.
- The page has a heading and paragraph.
- The page explains React in beginner words.
- The page mentions components, state, and routing.
- The page includes at least one SPA warning.
- The page works without a full browser reload during development changes.

---

27. Revision Checkpoint Questions and Answers

Question 1: Why is React more flexible than an opinionated framework?

React is more flexible because it mainly gives you tools for building user interfaces.

It does not force one complete app structure.

You can choose your own tools for:

- Routing
- Styling
- Data fetching
- Testing
- Project architecture

A framework usually makes more decisions for you.

Question 2: Name three issues SPAs can create if you do not handle them carefully.

SPAs can create:

1. Accessibility issues
2. Navigation and bookmark issues
3. SEO issues
4. Slow initial loading

Any three of these are acceptable.

Question 3: What does `npm install` do in a Vite project?

`npm install` installs the packages listed in the project.

These packages are called dependencies.

They are downloaded into the `node_modules` folder.

Simple answer:

```
npm install gets the project dependencies ready so the app can run.
```

Question 4: What file do you usually edit first after opening a Vite React starter?

You usually edit:

```
src/App.jsx
```

This file contains the main starter component.

---

28. Sprint 1 Key Vocabulary

React

A JavaScript library for building user interfaces.

User Interface

The visible and interactive part of an app or website.

Component

A small reusable piece of UI.

Library

A toolset that gives you functions or features, while you control the structure.

Framework

A bigger tool system that gives you features and usually controls more of the app structure.

SPA

Single Page Application.

A web app that loads one main HTML page and changes the view with JavaScript.

Vite

A modern frontend build tool used to create and run React projects.

npm

Node package manager.

Used to create projects, install dependencies, and run scripts.

Dependency

A package your project needs to work.

`node_modules`

The folder where installed dependencies are stored.

`package.json`

A project file that stores scripts, dependencies, and project information.

Development Server

A local server that lets you view and test your app while building it.

---

29. Sprint 1 Mental Models

React Mental Model

```
Break the UI into components.
Describe what each component should show.
When data changes, React updates the UI.
```

SPA Mental Model

```
Load one main HTML page.
Use JavaScript to change the visible screen.
Avoid full page reloads during navigation.
```

Vite Mental Model

```
Use Vite to quickly create and run a React project locally.
```

---

30. Common Beginner Mistakes

Mistake 1: Thinking React Is a Full Framework

React is not usually considered a full framework by itself.

React mainly focuses on UI.

Frameworks like Next.js add more full-app features.

Mistake 2: Thinking SPAs Have No Problems

SPAs can feel fast, but they can create issues with:

- SEO
- Accessibility
- Navigation
- Initial loading

Mistake 3: Running the Wrong Vite Command

Wrong:

```
npm create vite my-react-app template react
```

Correct:

```
npm create vite@latest my-react-app -- --template react
```

Mistake 4: Running `npm dev`

Wrong:

```
npm dev
```

Correct:

```
npm run dev
```

Mistake 5: Editing the Wrong File First

For this sprint, start with:

```
src/App.jsx
```

---

31. Sprint 1 Self-Check

You are ready to move on from Sprint 1 if you can do these things:

- Explain what React is in simple words.
- Explain what a component is.
- Explain the difference between a library and a framework.
- Explain why React is called a UI library.
- Explain why Next.js is called a framework.
- Explain what an SPA is.
- Describe how a traditional website loads pages.
- Describe how an SPA changes views.
- Name at least three SPA risks.
- Explain what Vite does.
- Create a React project using Vite.
- Run `npm install` and explain what it does.
- Run `npm run dev` and explain what it does.
- Open the app in the browser.
- Edit `src/App.jsx`.
- Build a simple React Roadmap page.
- Fix the broken Vite setup command.

---

32. Final Sprint 1 Summary

React is a JavaScript library for building user interfaces.

React helps developers build apps using reusable pieces called components.

React is flexible because it mainly focuses on the UI layer and lets you choose other tools for routing, styling, testing, and app structure.

An SPA, or Single Page Application, loads one main HTML page and uses JavaScript to update the visible screen without full page reloads.

SPAs can feel fast, but they need care because they can create accessibility, navigation, SEO, and first-load performance problems.

Vite is a modern frontend tool that helps you create and run React projects quickly.

The main setup commands are:

```
npm create vite@latest my-react-app -- --template react
cd my-react-app
npm install
npm run dev
```

The first file you usually edit in a Vite React starter is:

```
src/App.jsx
```

Your Sprint 1 practical goal is simple:

```
Create a Vite React app, edit App.jsx, run it in the browser, and explain what
React, SPAs, and Vite do.
```

---

33. Mini Revision Sheet

React

React builds user interfaces using components.

Component

A reusable piece of UI.

Library

Gives tools. You control the structure.

Framework

Gives tools and a larger structure.

SPA

One main HTML page. JavaScript changes the view.

SPA Benefits

Smooth page changes and fewer full reloads.

SPA Risks

Accessibility, navigation, SEO, and slow initial loading.

Vite

Creates and runs a modern React project.

Main Command

```
npm create vite@latest my-react-app -- --template react
```

Run the App

```
cd my-react-app
npm install
npm run dev
```

First File to Edit

```
src/App.jsx
```

---

34. Practice Prompt

After finishing Sprint 1, explain this out loud:

```
React is a UI library. It helps me build reusable components. An SPA loads one main HTML page and changes the screen with JavaScript instead of doing full page reloads. Vite helps me create and run a React project. To start, I use npm create vite@latest, install dependencies with npm install, run the app with npm run dev, and edit src/App.jsx first.
```

If you can explain that clearly, Sprint 1 is complete.